

Detecting Cloud Anomaly via Broad Network-Based Contrastive Autoencoder

Guoxiang Zhong^{1b}, Fagui Liu^{1b}, *Member, IEEE*, Jun Jiang^{1b}, *Member, IEEE*,
Bin Wang^{1b}, Xi Yao, and C. L. Philip Chen^{1b}, *Fellow, IEEE*

Abstract—Anomaly detection is indispensable for achieving higher availability and reliability in the cloud computing. The traditional autoencoder-based method only models the historical normal samples and then identifies the current online anomaly samples by the fixed threshold of anomaly score. Although more advances have been made in recent years, two main challenges remain: (i) ignoring the historical anomaly samples, (ii) poor self-adaptive ability for online detection. To address the above challenges, we propose a unified detector, namely BroadCAE, which integrates autoencoder with contrastive learning and broad network. Specifically, the reconstruction loss is first replaced by contrastive loss, which equally formulates both normal and anomaly samples. These samples belonging to the same class become closer in a lower-dimensional space. Conversely, different classes of samples are far away from each other. Next, we apply the anomaly-score-based pseudo thresholds to train the dynamic threshold selection, which generates the threshold according to the coming sample. The broad network in dynamic threshold selection takes the place of the deep network, which overcomes catastrophic forgetting and adapts to new online samples. Finally, validation experiments are conducted on four benchmark datasets. Our BroadCAE outperforms the comparative baseline methods by averaging over 4% of the f1-score.

Index Terms—Anomaly detection, contrastive learning, broad network, cloud computing.

I. INTRODUCTION

WITH the wide application of cloud computing, the scales of cloud services and their users are dramatically increasing. For a better user experience, the cloud operation

Manuscript received 16 August 2023; revised 22 December 2023; accepted 10 January 2024. Date of publication 15 January 2024; date of current version 12 July 2024. This work was supported in part by the Guangdong Major Project of Basic and Applied Basic Research under Grant 2019B030302002, in part by the Science and Technology Major Project of Guangzhou under Grant 202007030006, in part by the Science and Technology Project of Guangdong Province under Grant 2021B111160001, and in part by the Major Key Project of PCL, China under Grant PCL2023A09. The associate editor coordinating the review of this article and approving it for publication was H. Lutfiyya. (Corresponding authors: Fagui Liu; Jun Jiang.)

Guoxiang Zhong, Xi Yao, and C. L. Philip Chen are with the School of Computer Science and Engineering, South China University of Technology, Guangzhou 510006, China (e-mail: cszhongguoxiang111@mail.scut.edu.cn; 201930330247@mail.scut.edu.cn; philip.chen@ieee.org).

Fagui Liu is with the School of Computer Science and Engineering, South China University of Technology, Guangzhou 510006, China, and also with the Department of New Networks, Peng Cheng Laboratory, Shenzhen 518000, China (e-mail: fgliu@scut.edu.cn).

Jun Jiang is with the College of Information Science and Technology, Nanjing Forestry University, Nanjing 210037, China (e-mail: junjiang@ieee.org).

Bin Wang is with the Department of New Networks, Peng Cheng Laboratory, Shenzhen 518000, China (e-mail: wangb02@pcl.ac.cn).

Digital Object Identifier 10.1109/TNSM.2024.3353772

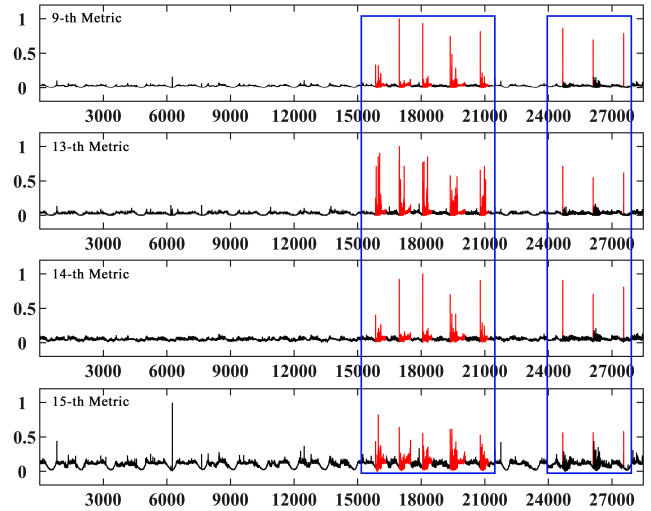


Fig. 1. Faults of the server cluster in SMD benchmark dataset. The red metric curves in blue boxes indicate the faults. These faults are described as anomalies of 9-th, 13-th, 14-th, and 15-th metrics, which also are referred to as root causes.

system must continuously improve the availability and reliability of cloud services [1], [2]. However, accidental faults, such as network delay [3], [4] and resource hog [5], [6], frequently pose a substantial threat to the system running. These faults cause the anomalous behaviors of the related monitoring metrics [7], [8], which is shown in Fig. 1. The operation system always discovers the faults by detecting the anomaly of running data and then starts the corresponding fault tolerance solutions [9], [10], [11]. In other words, anomaly detection is the first step to actively removing the faults [12], [13]. The powerful detection result maintains a solid foundation of higher availability and reliability in cloud [14], [15].

The autoencoder-based anomaly detection algorithm combines the robust neural network to mine the intrinsic patterns, which has significantly been concerned [16], [17], [18], [19]. More recent literature has also demonstrated that the encoder-decoder architecture outperforms traditional methods by a large margin, such as Bagel [20] and USAD [21]. The autoencoder-based method adopts the historical normal data to train the representation network. The reconstruction error provided by this representation is regarded as an anomaly score. The online testing data with a larger anomaly score

indicates the presented anomaly. More proposed works focus on optimizing the above-mentioned procedures [22], [23]. However, there still exist two main challenges, which are in the following:

- The first challenge is related to feature representation, which ignores the historical anomaly samples. For universal anomaly detection applications, normal samples constitute the majority of the training sample owing to the higher overhead of labeling scarce anomaly samples. The feature representation rebuilds the normal sample, which only formulates the intra-class features. Both intra-class features of anomalies and inter-class features between two kinds of samples are absent. The anomaly detection in the cloud is a special case with complete running documents [11], [24], [25]. The recorded anomalous events are abundant, which can provide feature representation with adequate and cheap labeled anomaly samples.
- The second challenge comes from poor self-adaptive ability in online detection. Cloud resource utilization is in a constant state of change due to various practical requirements. For example, if the computing-intensive task is uploaded, more computing nodes are in need. The CPU available percentage becomes lower. Subsequently, the cloud user deploys network-intensive tasks. The network byte sent rate will be higher. In the meanwhile, the metrics about computing return to normal. Without the self-adaptive ability, anomaly detection identifies the lower CPU available percentage and higher network byte sent rate as anomaly behaviors.

In this paper, we overcome these limitations and propose an improved autoencoder framework BroadCAE, which integrates contrastive learning and broad network. For the first challenge, contrastive loss is introduced to formulate the latent variable in the autoencoder. The anomaly samples in an embedding space are closer to each other as possible, which is the same operation for normal samples. Inversely, the training samples from different classes are separated. The feature representation extracts both intra-class and inter-class features, which further optimizes the margin between normal and anomaly samples. To tackle the second challenge, the BroadCAE incorporates the broad network module to develop dynamic threshold selection. We first apply the anomaly scores to calculate the pseudo thresholds. Specifically, the anomaly bias shrinks the anomaly scores of anomalies as their pseudo thresholds, while the normal bias amplifies the anomaly scores of normal samples to develop the normal-sample thresholds. Thus, the violation of the actual label would be minimal. Then we attain the threshold pattern by feeding the pseudo thresholds and initial samples into the broad network. In the online testing stage, this trained broad learning model outputs the individual threshold according to the coming sample. A portion of the network weight is updated while the actual label of the testing sample is acquired, which fits the need for the new detection setting.

The major contributions of our proposed BroadCAE are:

- The BroadCAE innovates anomaly samples into the traditional normal-only encoder-decoder framework, which develops both inter-class and intra-class features

to empower the representation of cloud anomaly detection.

- The BroadCAE adopts the broad network without deep structure to dynamically generate thresholds in the light of the coming samples, which provides a different perspective to refine the anomaly detection system.
- In validation experiments, our BroadCAE outperforms other comparative algorithms, which demonstrates the applicability of BroadCAE for cloud anomaly identification. The open-source codes¹ will continue to motivate the research in cloud anomaly detection.

The rest of the paper is organized as follows. Section II reviews the BroadCAE's related work. The design details of BroadCAE are discussed in Section III, containing problem formulation, BroadCAE's overview, contrastive autoencoder, broad network-based dynamic threshold selection, and BroadCAE's pseudo code. The experiment in Section IV validates the effectiveness of BroadCAE by answering three research questions. Finally, the conclusion is presented in Section V.

II. RELATED WORK

A. Background of Cloud Anomaly

The anomaly in the cloud is a running behavior that deviates from the normal system setting on account of fault. According to different fault locations, cloud anomaly can be divided into internal and external behavior. The external factor caused by other services leads to an external anomaly. For example, the cloud SaaS application downloads a large file, which brings about network congestion [26]. Besides, cyber attack [27] is the most common external fault. Internal processes induce internal anomalies, such as authentication failure and network packet loss. The operation system uses several reactive and proactive fault tolerance methods, such as auto-scaling [5], to remove the presented fault [28], which avoids further affecting the regular cloud services. Before that, the operation system still needs to consider identifying the fault through the anomaly. They always analyze the anomaly state by monitoring metric, log, and trace. Particularly, the monitoring metric is easier to embed and reveal the most comprehensive data pattern. In this paper, we focus on the monitoring-metric-based anomaly detection algorithm.

B. Cloud Anomaly Detection Algorithm

In the early stage, the cloud operation system primarily adopts keyword-based [29] and statistics-based [30] anomaly detection methods. The keyword-based method identifies the anomaly recorded in the log by matching the related words, such as ERROR and FAILURE [31]. The statistics-based methods focus on several key performance indicators [32]. If their values exceed a certain setting threshold, an alert is triggered. The keyword-based and statistics-based methods are only suitable for anomalies with relatively simple patterns.

With the development of artificial intelligence operations in the cloud, more machine-learning-based anomaly detections

¹<https://github.com/zhongguoxiang/BroadCAE>

TABLE I
THE COMPARISON OF RELATED ALGORITHMS AND OUR BROADCAE

Algorithm \ Item	Formulate two-class samples	Mine deep-level features
LOF	✓ ¹	✗ ¹
OC-SVM	✗	✗
AE-based methods	✗	✓
BroadCAE	✓	✓

¹ The ✓ and ✗ indicate the advantage and disadvantage, respectively.

are deployed [31], [33], [34], [35]. They always depend on the distribution divergence between normal and anomaly samples. For instance, the local outliers factor (LOF) [36] believes that the distribution density of normal is higher than the anomaly sample's. The one-class support vector machine (OCSVM) [37] considers that all normal samples are in a hyper-sphere. However, these methods have limited feature modeling ability and are not robust enough. As a result, they are even invalidated when the large-scale samples do not follow the corresponding assumed distribution.

At the same time, the cloud architecture is becoming larger. The coordination between cloud components steadily grows, leading to more monitoring data. It is difficult for the classic machine-learning-based method to meet the practical application requirements. The root cause is their weak modeling of mining data patterns. The autoencoder (AE) algorithm [38] is an effective solution that formulates massive historical normal sample data to obtain deep-level features. More potential patterns can be employed to identify complex cloud anomalies. Some proposed works improve the vanilla autoencoder, which has obvious advantages in terms of detection efficiency and accuracy. For instance, Donut [39] formulates the anomalies and missing points into the ELBO of variational autoencoder (VAE) [40], which outperforms the random-forest-based Opprentice [41]. As an improved method of Donut, Bagel [20] handles the time information based on the conditional VAE (CVAE) [42]. The superiority of Bagel in detection accuracy is also significant. Both Donut and Bagel modify the loss function. In addition, other approaches concentrate on developing the network architecture. The VAE-LSTM [43] introduces long short-term memory (LSTM) into the backbone network of VAE for modeling metric-based time series. The UASD [21] inherits the encoder-decoder architecture and adds one more decoder as a discriminator for adversarial training. The Anomaly Transformer [22] and TranAD [23] integrate the attention mechanism [44], which enjoys long-term series modeling.

As displayed in Table I, our BroadCAE goes beyond the previous autoencoder-based algorithm and considers the anomaly sample for formulating more features. It is also the first time to unify anomaly and normal sample patterns under the autoencoder representation.

C. Broad Network

The broad network [45] is an improved single-layer feed-forward neuron model. Without the deep architecture, the broad network still has universal approximation properties [46] and is much faster with credible classification accuracy. The weight in the broad network is solved by the pseudoinverse rather than gradient-descent-based methods used in deep networks. The generalization ability of the broad network is improved through the broad expansion. More broad network variants are proposed. For example, fuzzy broad learning system [47] introduces the Takagi-Sugeno fuzzy approach into the feature node. Stacked broad learning system [48] develops the incremental flatted framework to deep network, which outperforms the deep residual network in experimental comparison. Although the broad network is still in the early developing stage, some application researches continuously model this novel network. The broad network combines the graph neural network to achieve the emotion recognition [49]. The semi-supervised broad network is built for spammer detection in social communication networks [50].

Unlike the above broad network application, our BroadCAE unifies the deep-network-based autoencoder and broad network for cloud anomaly detection. For the broad network, we apply the increment of input samples, which is designed for self-adaptive ability in the online setting.

III. METHODOLOGY

A. Problem Formulation

Suppose there are several metrics for monitoring cloud components. We attain their labeled history running data as

$$\begin{aligned} D &= \{(x_i, y_i) | i = 1, \dots, N\}, \\ x_i &= [x_{i1}, x_{i2}, \dots, x_{id}]^T \in \mathbb{R}^d, \\ y_i &\in \{+1, -1\}, \end{aligned} \quad (1)$$

where x_i represents the i -th sample in D and y_i is the label of x_i . The N is the number of samples in D . The d indicates the number of monitoring metrics. The “+1” and “−1” are the label marks of normal and anomaly samples, respectively.

The autoencoder-based anomaly detection algorithm regards D as the training dataset and formulates the feature representation with D . The representation of this anomaly detector contains encoder f_e and decoder f_d . For the normal sample $(x_i, y_i = +1) \in D$, the encoder f_e transforms x_i into latent variable Z .

$$\begin{aligned} z_i &= f_e(x_i), z_i \in Z, \\ y_i &= +1, \end{aligned} \quad (2)$$

where z_i is the latent variable corresponding to x_i . Then, the decoder f_d inversely transforms z_i into x'_i :

$$x'_i = f_d(z_i), x'_i \in \mathbb{R}^d, \quad (3)$$

where x'_i is the reconstruction of x_i . What is more, both f_e and f_d are in the form of deep neural networks. We optimize

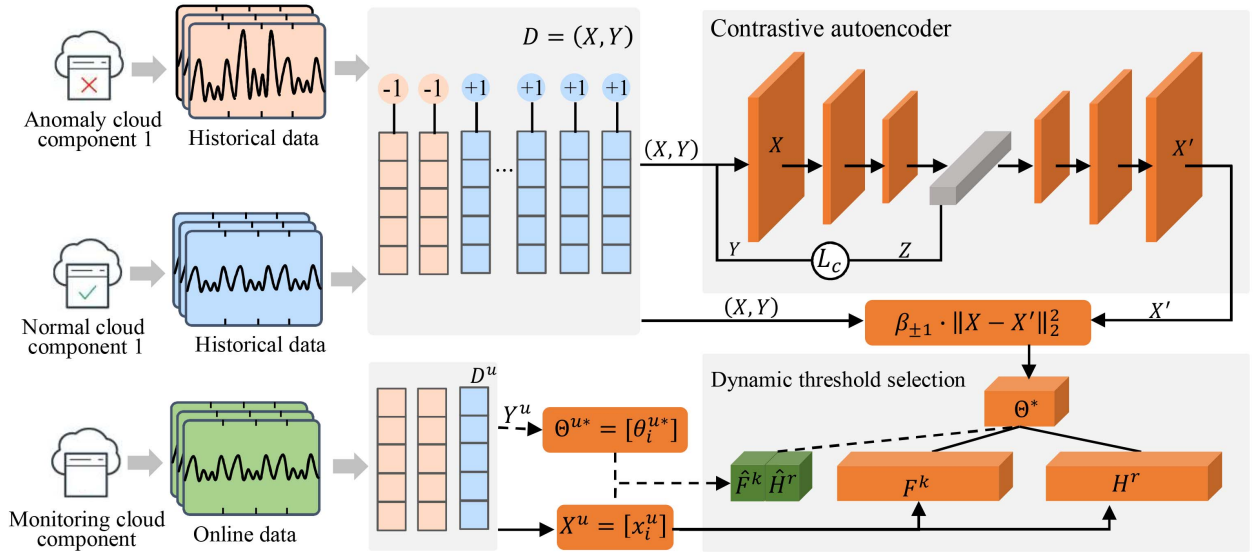


Fig. 2. The overview of BroadCAE. The training-use historical dataset includes the labeled normal and anomaly samples in cloud computing. Our BroadCAE consists of contrastive-learning-based CAE and broad network-based dynamic threshold selection. The CAE improves the feature representation by adding anomaly samples. The dynamic threshold selection has the powerful self-adaptive ability for online detection. The update procedure (---) empowers the broad network.

the objective representation by minimizing the following reconstruction loss function L_r :

$$L = L_r(X, X') = \frac{1}{N} \sum_{i=1}^N \|x_i - x'_i\|_2^2, \quad (4)$$

where $\|\cdot\|_2^2$ denotes the metric of similarity in Euclidean distance. Because the normal sample is always far from the anomaly ones, the reconstruction error of the normal sample is smaller than the anomaly's. This divergence in reconstruction error is the core of achieving the autoencoder-based anomaly detector. The reconstruction error is referred to as the anomaly score. Given the online testing sample $x_i^u \in D^u$ where D^u represents the online testing dataset, $\hat{y}_i^u$ is the predicted label of x_i^u . The autoencoder-based anomaly detector outputs the predicted label as

$$\hat{y}_i^u = \begin{cases} +1, & AS_i^u \leq \theta; \\ -1, & AS_i^u > \theta; \end{cases} \quad (5)$$

$$AS_i^u = \|x_i^u - f_d[f_e(x_i^u)]\|_2^2,$$

where θ is a threshold and AS_i^u is the anomaly score of x_i^u .

Challenges: (i) Ignoring the historical anomaly samples in training the feature representation. From Equation (2), only the normal sample is formulated by f_e . Actually, anomaly samples also can enlarge the reconstruction divergence. In detail, The inter-class feature between anomaly and normal samples requires that the latent variable of anomaly samples should be separated as possible. The intra-class feature from anomaly samples distinguishes the overlapping normal samples, which enhance the robustness of f_e and f_d . (ii) Poor self-adaptive ability for online detection. In Equation (5), the threshold θ is fixed, which does not enjoy dealing with the online continual change samples. A small portion of anomaly samples will be detected if the threshold value is larger enough. In contrast,

if the threshold value is too small, the predicted anomaly samples may contain more normal ones. The autoencoder-based methods need the self-adaptive threshold selection, which offers the dynamic threshold.

B. Overview

This paper focuses on the above challenge and proposes the BroadCAE that introduces contrastive learning and broad network. The overview of BroadCAE is shown in Fig. 2. The historical running data of normal and anomalous cloud components are collected. The data preprocessing includes padding and normalization, which is omitted in the overview.

The contrastive autoencoder (CAE) innovates contrastive learning into the autoencoder. The autoencoder reconstruct $X = [x_i]_{1 \times N}$ as $X' = [x'_i]_{1 \times N}$. The feature representation ignores the anomaly samples, which ascribes to the dimension reduction transformation in the encoder. For the latent variable $Z = [z_i]_{1 \times N}$, the divergence of normal and anomaly samples should be more obvious. Thus, we add the contrastive loss L_c to the latent variable. The contrastive loss uses the constraint about sample label $Y = [y_i]_{1 \times N}$ and latent variable Z .

The broad network combines dynamic threshold selection, which is designed for self-adaptive online detection. The combination of anomaly score $\|X - X'\|_2^2$ and sample label Y calculates the pseudo label Θ^* for threshold selection. The predicted threshold Θ^u is provided while the online coming sample $X^u = [x_i^u]$ is fed into the trained broad network. Furthermore, the relationship between Θ^u and anomaly score of X^u decides the predicted label $\hat{Y}^u = [y_i^u]$. The broad network weight can be updated when the actual label Y^u of X^u is acquired. It is noteworthy that only the network weights related to the online coming samples are retrained. The broad network avoids updating the whole network weights, which is a great substitute for the deep network.

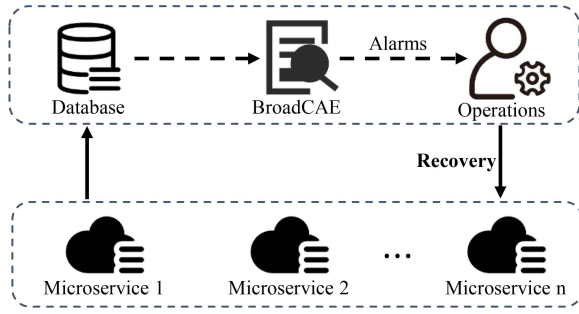


Fig. 3. Potential deployment of BroadCAE in microservice.

For example, Fig. 3 describes our BroadCAE, which is deployed into the cloud-based microservice system. The monitoring data of each microservice is collected and delivered to the database. The detector BroadCAE identifies the anomaly data from the database and transmits the alert signal to operations.

C. Contrastive Autoencoder

The general autoencoder-based method builds up the representation by training with normal samples. The extracted features in the representation can be regarded as intra-class ones of normal samples. The anomaly samples are discovered, which follows the principle “normal is not anomalous”. In other words, the normal-sample intra-class features provide the manifold of normal. The testing sample far away from this manifold is not normal. The approaches only model the seen or occurred normal samples. However, the novel normal pattern constantly emerges in cloud operation. It is unachievable to model all types of normal samples under the evolving cloud. The representation manifold only the training normal samples, which will gradually lose efficacy in the future. Besides, in the cloud environment, the deployed cloud component needs more preliminary tests. The anomaly events supply more anomaly samples to optimize the autoencoder. The anomaly-sample intra-class features indicate whether the sample is not normal. The inter-class features between normal and anomaly samples lead to the margin for dividing the samples. The additional intra-class and inter-class features enhance the representation. Thus, it is necessary to incorporate anomaly samples into the representation. The CAE in our BroadCAE introduces contrastive loss for formulating the anomaly samples. The contrastive loss focuses on the lower-dimensional latent variable, which is compacted by the encoder f_e . Suppose that $(x_i, y_i) \in D$ and $(x_j, y_j) \in D$ are given, their latent variables are

$$z_i = f_e(x_i), z_j = f_e(x_j). \quad (6)$$

The contrastive loss is

$$L_c = \frac{1}{2N^2} \sum_{i,j=1}^N \left((1 - y'_{ij}) S_{ij}^2 + y'_{ij} \left[\max(0, m - S_{ij}^2) \right] \right), \quad (7)$$

where $S_{ij} = \|z_i - z_j\|_2^2$ is the Euclidean-distance similarity between z_i and z_j . The $\max(\cdot, \cdot)$ returns the largest value. The $m > 0$ is the classification margin about distance. In our experiments, $m = 2$. The pairs of samples far away from this

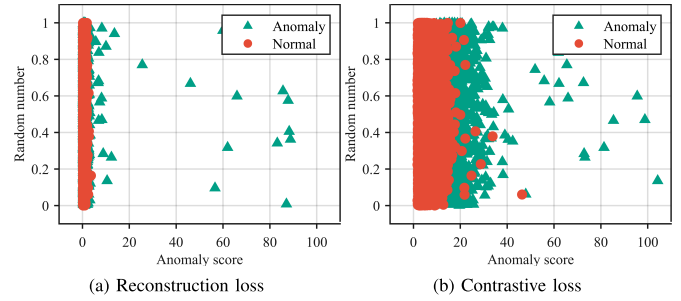


Fig. 4. Distribution of anomaly score using different loss functions. In (a), the majority of anomaly samples are covered by normal samples, which indicates that the anomaly-score divergence is too smaller. However, in (b), more anomaly samples have larger anomaly scores than normal samples, which contributes to anomaly identification.

margin do not contribute to the contrastive loss. If x_i and x_j are in the same classes, we annotate the pseudo label $y'_{ij} = 0$. While they belong to different classes, the pseudo label is shown as $y'_{ij} = 1$.

$$y'_{ij} = \begin{cases} 0, & y_i = y_j \\ 1, & y_i \neq y_j \end{cases}, \quad Y' = [y'_{ij}]_{N \times N}. \quad (8)$$

In the first part of L_c , the f_r strains every nerve to pull closer points, which are from the same class. The intra-class features of normal and anomaly samples are extracted. The second part aims at separating samples into different classes by a large margin. This operation enjoys the inter-class features between normal and anomaly samples.

During the training stage, all labeled training samples are fed into f_e and f_d , which are optimized by contrastive loss rather than reconstruction loss.

$$L = L_c(X, Y). \quad (9)$$

On the one hand, both contrastive loss and reconstruction loss can formalize the normal-sample intra-class features. In the meanwhile, beyond the reconstruction loss, the contrastive loss develops the anomaly-sample intra-class and anomaly-normal inter-class features. On the other hand, the latent variable divergence between normal and anomaly samples achieves a more powerful effect than reconstruction divergence, using the identical decoder. For example, Fig. 4 displays the anomaly score of the first SMD subset. The contrastive loss enlarges the anomaly score of anomaly samples.

D. Dynamic Threshold Selection Based on Broad Network

Suppose that $G(\cdot)$ is the dynamic threshold selection in our BroadCAE. As for different inputs x_i^u and x_j^u , their output thresholds are also different.

$$\begin{aligned} \theta_i^u &= G(x_i^u), \theta_j^u = G(x_j^u), \\ \theta_i^u &\neq \theta_j^u. \end{aligned} \quad (10)$$

For the self-adaptive ability, $G(\cdot)$ should be update to reflect the online coming sample. The classical deep-network-based threshold selection algorithm repeats the training procedure to update the network weights. After several above replications, the knowledge about the previous training samples

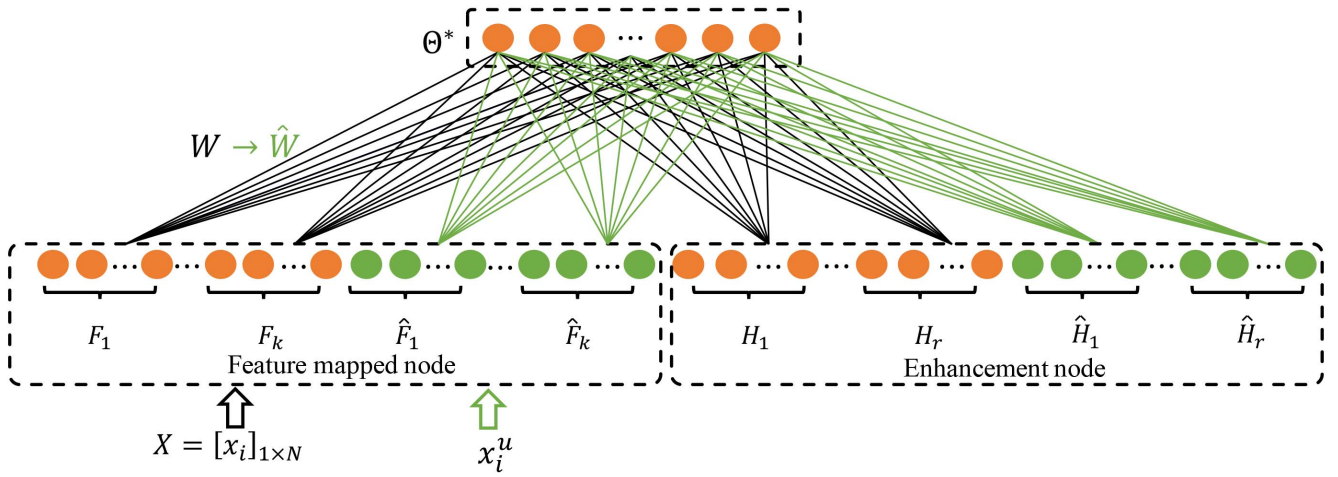


Fig. 5. The overview of broad network. This flattened network has only one input layer and one output layer. There are k feature mapped nodes and r enhancement nodes in the input layer. The network weight is solved by pseudoinverse. The increment of x_i^u is in green.

will disappear, which is called as catastrophic forgetting [51]. The $G(\cdot)$ addresses this problem by incorporating the broad network, not the deep network. As shown in Fig. 5, the broad network consists of two main components: feature mapped node and enhancement node. Given $X = [x_i]_{1 \times N}$ as the training sample, the k -th feature mapped node projects X as

$$F_k = \phi(XW_{f_k} + \beta_{f_k}), \quad (11)$$

where W_{f_k} and β_{f_k} are the initial weights and bias of F_k , respectively. The ϕ is a mapping function. The $F^k = [F_1, \dots, F_k]$ denotes all the first k -th feature mapped nodes in the form of concatenation. The output of F^k is mapped by enhancement nodes. The r -th enhancement node is

$$H_r = \xi(F^k W_{h_r} + \beta_{h_r}), \quad (12)$$

where W_{h_r} and β_{h_r} are the initial weights and bias of H_r , respectively. The ξ is a mapping function. The first r -th enhancement nodes are also in the form of concatenation.

$$H^r = [H_1, \dots, H_r]. \quad (13)$$

The broad network has k feature mapped nodes and r enhancement nodes, which is formulated as

$$\begin{aligned} \Theta^* &= [F_1, \dots, F_k \mid H_1, \dots, H_r] W \\ &= [F^k \mid H^r] W, \end{aligned} \quad (14)$$

where $\Theta^* = [\theta_i^*]_{1 \times N}$ is the threshold training label of X in $G(\cdot)$. The W is the network weight for the broad network. Different from the deep network, the broad network attains the optimal W through Moore-Penrose generalized inverse theory.

$$W = [F^k \mid H^r]^+ \Theta^*, \quad (15)$$

where $[F^k \mid H^r]^+$ denotes the pseudoinverse of $[F^k \mid H^r]$. Suppose that $A_k^r = [F^k \mid H^r]$, we have

$$(A_k^r)^+ = \lim_{\lambda \rightarrow 0} (\lambda I + A A^T)^{-1} A^T. \quad (16)$$

As for $\Theta^* = [\theta_i^*]_{1 \times N}$, we can not directly attain from the previous problem setting. Instead, we develop the supervised pseudo threshold.

$$\begin{aligned} \theta_i^* &= \begin{cases} \beta_{+1} \cdot AS_i, & y_i = +1, \\ \beta_{-1} \cdot AS_i, & y_i = -1, \end{cases} \\ AS_i &= \|x_i - x_i'\|_2^2, \end{aligned} \quad (17)$$

where β_{-1} and β_{+1} are anomaly bias and normal bias, respectively.

$$0 < \beta_{-1} < 1, \beta_{+1} \geq 1. \quad (18)$$

In this solution, it is not hard to show

$$\begin{aligned} \min \quad & \sum_{i=1}^N |y_i - y_i^*| = 0, \\ \text{s.t.} \quad & y_i^* = \begin{cases} +1, & AS_i \leq \theta_i^*, \\ -1, & AS_i > \theta_i^*, \end{cases} \end{aligned} \quad (19)$$

where y_i^* is the label of x_i provided by pseudo threshold. Thus, the generative pseudo threshold is practicable.

After finishing the training of broad network, the dynamic threshold selection $G(\cdot)$ will enter the online detection to identify the presented anomaly. Given the online coming sample $x_i^u \in D^u$, the $G(\cdot)$ follows

$$\begin{aligned} \theta_i^u &= G(x_i^u), \\ \hat{y}_i^u &= \begin{cases} +1, & AS_i^u \leq \theta_i^u, \\ -1, & AS_i^u > \theta_i^u. \end{cases} \end{aligned} \quad (20)$$

while the predicted label $\hat{y}_i^u$ is equal to the actual label y_i^u , the accurate prediction offer the correct sample pattern. The broad network in $G(\cdot)$ can be updated. The increments of the feature mapping node and enhancement node are

$$\begin{aligned} \hat{F}_k &= \phi(x_i^u W_{f_k} + \beta_{f_k}), \\ \hat{H}_r &= \xi(\hat{F}^k W_{h_r} + \beta_{h_r}), \end{aligned} \quad (21)$$

Algorithm 1 Training Algorithm of BroadCAE

Input: Encoder f_e ; Decoder f_d ; Training sample X ; Training label Y ; Number of feature mapped node k ; Number of enhancement node r ; Random mapping ϕ and ξ ; Iteration limitation N_e .

Output: Optimized encoder f_e ; Optimized decoder f_d ; Broad network weight W ; Network pseudoinverse $(A_k^r)^+$.

- 1: Initialize the network weights of f_e and f_d ;
- 2: Initialize the weights and biases of ϕ and ξ ;
- 3: $n \leftarrow 1$;
- 4: **repeat**
- 5: Feed X into f_e and f_d ;
- 6: Calculate the contrastive loss L_c ;
- 7: Optimize f_e and f_d with loss function $L = L_c(X, Y)$;
- 8: $n \leftarrow n+1$;
- 9: **until** ($n = N_e$).
- 10: Generate the pseudo threshold Θ^* ;
- 11: Feed X into feature mapped node F_k ;
- 12: Project $F^k = [F_1, \dots, F_k]$ by enhancement node H_r ;
- 13: Deduce the pseudoinverse $(A_k^r)^+$ and network weight W .

where $\hat{F}^k = [\hat{F}_1, \dots, \hat{F}_k]$ are the first k -th incremental mapping features of x_i^u . The A_k^r in the initial network is transformed as

$$\hat{A}_k^r = \begin{bmatrix} A_k^r \\ \hat{A}^T \end{bmatrix}, \quad (22)$$

where $\hat{A}^T = [\hat{F}^k | \hat{H}^r]$ and $\hat{H}^r = [\hat{H}_1, \dots, \hat{H}_r]$. The update pseudoinverse is

$$(\hat{A}_k^r)^+ = [(A_k^r)^+ - BD^T | B], \quad (23)$$

where $D^T = \hat{A}^T (A_k^r)^+$,

$$B^T = \begin{cases} (C)^+ & , C \neq 0, \\ (1 + D^T D)^{-1} (A_k^r)^+ D & , C = 0, \end{cases} \quad (24)$$

and $C = \hat{A}^T - D^T A_k^r$. The updated network weight is

$$\hat{W} = W + (\theta_i^{u*} - \hat{A}^T W) B, \quad (25)$$

where θ_i^{u*} is the pseudo threshold of x_i^u . From the Equation (25), we can apply the initial network weight W and initial pseudoinverse A_k^r to calculate the $\hat{W}$. The previous knowledge about W and A_k^r in the initial network are retained, which avoids catastrophic forgetting. In the meanwhile, the new pattern about the new x_i^u is described by $\hat{A}^T$, which adapts quickly to the new developing environment.

E. Implementation

The implementation of our BroadCAE is composed of training (Algorithm 1) and testing (Algorithm 2) blocks. All of the input of these two Algorithms can be directly calculated in the neural network. In Algorithm 1, we consider the feature representation of CAE from Line 3 to Line 9. There are N^2 pairs training samples. The quadratic computation about similarity in L_c causes the time complexity of CAE to be

Algorithm 2 Testing Algorithm of BroadCAE

Input: Optimized encoder f_e , Optimized decoder f_d ; Broad network weight W and pseudoinverse $(A_k^r)^+$; Random mapping ϕ and ξ ; Testing sample X^u .

Output: Predicted label $\hat{Y}^u$.

- 1: Reconstruct X^u through f_e and f_d ;
- 2: Compute the anomaly score of X^u ;
- 3: Calculate the threshold of X^u ;
- 4: Attain the predicted label Y^u
- 5: **while** $\hat{Y}^u = Y^u$ **do**
- 6: Update pseudoinverse $(A_k^r)^+$ as $(\hat{A}_k^r)^+$;
- 7: Update network weight W as $\hat{W}$;
- 8: **end while**

TABLE II
THE CHARACTERISTICS OF FOUR EXPERIMENTAL DATASETS

Dataset	MBD	EMOS	SMD	ASD
# of Sample	8640	23947	708420	51840
# of Metric	130	164	38	19
# of Anomalous rate	6.55%	8.98%	4.16%	4.61%

$\mathcal{O}(N^2)$. The dynamic threshold selection is between Line 10 and Line 13. The dot-product for computing the pseudoinverse makes the time complexity of dynamic threshold selection $\mathcal{O}(N^2)$. Thus, our BroadCAE achieves the quadratic time complexity. In Algorithm 2, our goal is to predict the label of X^u . Besides, the threshold selection adapts to the new coming samples by updating the network weight and pseudoinverse, which is presented from Line 5 to Line 8.

IV. EXPERIMENT

We first introduce the experimental setting, including datasets, performance metrics, and experimental details. Then, the following research questions (RQ) about validating our BroadCAE are answered by a series of experiments. Their experimental results and analysis are displayed.

RQ1: How about the overall performance of BroadCAE in cloud anomaly detection?

RQ2: How about the impact of parameters in BroadCAE?

RQ3: How about the effectiveness of each component in BroadCAE?

A. Datasets

We conduct the experiments on four benchmark datasets, including MBD [52], EMOS, SMD [53], and ASD [54]. These datasets are collected from different cloud environments. The first dataset is from a Hadoop-based distributed cluster. The second one is related to a benchmark microservice-based system.² The last two datasets correlate with the server in the cloud. The characteristics of these datasets are in Table II.

The MBD reflects the running behavior of five cluster nodes, which process the big data batch. To imitate the actual setting, the researchers randomly upload the large-scale

²<https://github.com/FudanSELab/train-ticket>

workload using HiBench.³ The faults of the CPU, network, and application are randomly injected into the file system. There are 26 monitoring metrics for each node, including CPU, disk, memory, network, and process.

The EMOS presents the status of 41 microservices, which service the train ticket booking application.⁴ The faults are injected by Chaosblade⁵ and iPerf.⁶ These faults include CPU hog, memory hog, disk input, disk output, DDOS delay, and packet loss. We monitor four representative metrics, including CPU usage, RAM usage, Net out, and Net in. All faults and metrics are used in some published works [5], [52]. For the purpose of research, we encourage anyone to use this dataset. More details are provided here.⁷

The SMD is provided by a commercial computing cluster, which includes 28 machines. During the 5-week-long period, the collected data consists of two subset components. The first half is used for training, and the remaining half is regarded as a testing subset. However, only the testing subset provides both the label and the dimensions of the anomaly. For performance evaluation, we adopt the testing subset in our experiments.

The ASD describes the status of 12 servers in a large Internet company. To avoid the concept drifts, the stable workloads run on each server during a 45-days-long period. We also utilize the testing subset of ASD. However, the ASD contains fewer metrics, which differs from the SMD. The monitoring metrics are related to CPU, memory, network, and virtual machine.

B. Performance Metrics

In general, recall (R), precision (P), and f1-score (F1) are used to evaluate anomaly detection algorithm performance. The R focuses on presenting the accuracy of identifying the goal samples. The P considers that the presented goal samples are detected as more as possible. The F1 is introduced to keep the balance between R and P. However, for more comprehensive evaluation [21], [53], we import the macro R ($\bar{R}$), macro P ($\bar{P}$), and macro F1 ($\bar{F1}$) in this paper.

$$\begin{aligned}\bar{R} &= \frac{1}{2} \sum R, \bar{P} = \frac{1}{2} \sum P, \\ \bar{F1} &= \frac{2 \cdot \bar{R} \cdot \bar{P}}{\bar{R} + \bar{P}}.\end{aligned}\quad (26)$$

C. Experimental Details

The BroadCAE is implemented in a Python program. Both encoder f_e and decoder f_d with deep architectures are employed through the Pytorch package. The f_e contains a 4-layer multi-layer perceptron (MLP), the size of output features of which are $d \rightarrow 1024 \rightarrow 512 \rightarrow 256$. The ReLU [55] and batch normalization [56] are added into each layer. The f_d is made up of a 3-layer MLP ($256 \rightarrow 128 \rightarrow d$) with batch normalization. The ReLU is also added into the first layer of f_d . The f_e and f_d are optimized by the ADAM optimizer [57]

³<https://github.com/Intel-bigdata/HiBench>

⁴<https://github.com/FudanSELab/train-ticket>

⁵<https://github.com/chaosblade-io/chaosblade>

⁶<https://iperf.fr/>

⁷<https://github.com/zhongguoxiang/BroadCAE>

TABLE III
THE CHARACTERIZATIONS OF EXPERIMENTAL COMPUTER SERVER

Item	Characterization
CPU	16×Intel(R) Core(TM) i7-10700 2.90GHz
Memory	64.00GB
Program	Python 3.7+Pytorch 1.6
Operate system	Windows 10 professional

with 10^{-3} initial learning rate and step-scheduler with step size of 0.5. We run 100 epochs, where the size of the batch is 256. The other hyper-parameters of our BroadCAE are listed in the following.

- Number of feature mapped nodes $k = 35$.
- Number of enhancement nodes $r = 500$.
- Anomaly bias $\beta_{-1} = 0.01$.
- Normal bias $\beta_{+1} = 2$.

For four datasets, we randomly split all samples into training and testing subsets with a ratio of 5:5. Then, we follow the above-mentioned parameters and repeat the identical procedure for one hundred independent runs. The mean of these replications can be regarded as the final experimental results. The comparative baselines also follow the same operations. Some comparative anomaly detection methods only output the anomaly score of the testing sample. The threshold selection is absent. We plot the ROC curve [58] and search for the best detection result.

The BroadCAE and other comparative anomaly detection algorithms are tested in the server computer, which equips with one NVIDIA GeForce RTX 3080. More details of experiment equipment are shown in Table III.

D. Research Questions

We conduct the experiments about three research questions. The experimental results and analysis should validate the effectiveness of BroadCAE.

RQ1: How about the overall performance of BroadCAE in cloud anomaly detection?

The RQ1 is designed to demonstrate whether BroadCAE can outperform the state-of-the-art baseline approaches in cloud anomaly detection. There are 12 comparative algorithms used in this experiment. Among them, the LOF, OCSVM, and iForest are proposed in the earlier period. With the rapid development of deep learning, autoencoder-based methods are more and more applied because of their strong generalization. The AE, VAE, and CVAE are the baseline algorithms. The Bagel are adapted from CVAE. The USAD and SaVAE-SR innovate the training style, which is inspired by the adversarial learning theory of GAN [62]. Both TranAD and Anomaly Transformer are developed from the improved autoencoder-based architecture Transformer [44]. The CAE-M utilizes the autoencoder-based architecture to mine the semantics features. These comparative algorithms are introduced as follows:

- LOF [36]: The local reachability density (LRD) about nearest neighbor samples is first released to measure the anomaly degree. The higher values of LDR, the less anomalous the sample is.

TABLE IV
THE OVERALL PERFORMANCE RESULTS FOR OUR BROADCAE AND OTHER COMPARATIVE METHODS IN FOUR BENCHMARK DATASETS

Algorithm	MBD			EMOS			SMD			ASD		
	$\bar{F}1^1$	$\bar{R}$	$\bar{P}$	$\bar{F}1$	$\bar{R}$	$\bar{P}$	$\bar{F}1$	$\bar{R}$	$\bar{P}$	$\bar{F}1$	$\bar{R}$	$\bar{P}$
LOF [36]	63.54	66.63	60.72	58.68	59.14	58.23	55.37	55.65	55.10	56.55	58.97	54.32
OCSVM [59]	59.28	66.01	53.79	61.25	68.00	55.72	66.44	76.07	58.97	55.35	59.61	51.67
iForest [60]	52.12	52.79	51.46	52.07	52.72	51.44	70.14	72.68	67.77	54.49	57.38	51.87
AE [38]	60.14	67.53	54.20	60.58	66.87	55.37	64.59	72.08	58.51	56.82	62.33	52.21
VAE [40]	60.40	68.02	54.32	59.72	64.00	55.99	68.18	78.34	60.36	56.60	61.93	52.11
CVAE [42]	59.76	66.82	54.06	59.45	64.40	55.21	65.35	73.45	58.85	56.61	61.98	52.10
Bagel [20]	68.69	54.37	93.25	63.15	51.16	82.50	74.24	60.45	96.16	67.29	56.29	83.62
USAD [21]	80.01	77.61	82.56	<u>84.06</u>	83.36	<u>84.77</u>	85.91	82.77	89.30	70.28	64.03	77.89
SaVAE-SR [14]	81.15	79.07	83.35	83.45	<u>84.94</u>	82.01	85.08	83.06	87.20	61.78	52.96	74.13
TranAD [23]	80.01	77.61	82.56	83.54	82.86	84.22	86.75	85.05	88.52	69.78	63.63	77.24
Anomaly Transformer [22]	75.37	74.97	75.77	80.85	81.01	80.68	<u>91.50</u>	<u>97.65</u>	86.08	58.97	51.13	69.65
CAE-M [61]	<u>82.37</u> ²	<u>80.54</u>	84.27	82.94	82.14	83.76	84.61	82.48	86.85	<u>70.54</u>	<u>64.17</u>	<u>78.32</u>
BroadCAE	82.87 ³	81.36	<u>84.44</u>	89.02	86.43	91.76	96.62	98.40	<u>94.90</u>	76.99	79.70	74.46

¹ The higher values of $\bar{F}1$, $\bar{R}$, and $\bar{P}$ (as %) indicate the better performance.

² The second highest values are underlined.

³ The highest values are in bold type.

- OCSVM [59]: The hyper-sphere is optimized in the training dataset. The testing sample outside of the hyper-sphere is anomalous.
- iForest [60]: From its authors' view, the anomaly is easier to be isolated. The iForest adopts isolation instead of distance and density to identify anomaly samples.
- AE [38]: Both the encoder and the decoder with a deep network rebuild the samples. The back-propagation algorithm with reconstruction loss optimizes the network weight. The reconstruction error presents the anomaly score.
- VAE [40]: Based on AE, the latent variable is described in the form of Gaussian distribution. The reconstruction probability of KL divergence is added into the ELBO loss function and further indicates how anomalous the testing sample is.
- CVAE [42]: Being distinguished from AE and VAE, CVAE formulates the model about the relationship between the input latent variable, input sample, and its label. In fact, the CVAE is one of the semi-supervised encoder-decoder-based methods.
- Bagel [20]: Compared to Dount [39], Bagel includes time information into CVAE and enhances the robustness in detecting the time-related anomaly.
- USAD [21]: In the encoder-decoder architecture, it imports the adversarial training by adding a decoder block. At the same time, it also avoids collapse and non-convergence phenomena.
- SaVAE-SR [14]: It draws its strength from combining spectral residual technique and self-adversarial training manner, which still follow the framework of VAE.
- TranAD [23]: The Transformer [44] is introduced. The vanilla attention mechanism is designed for

time-dependency relationship. Both the self-condition approach and adversarial learning stabilize the training procedure.

- Anomaly Transformer [22]: The Transformer architecture is improved by developing a new anomaly-attention mechanism. The association discrepancy and minimax strategy enlarge the divergence between normal and anomaly samples.
- CAE-M [61]: The deep convolutional characterization network is employed as a feature extractor. The attention-based bidirectional LSTM and auto-regression provide the final predicted result.

Table IV presents the overall performance results of our BroadCAE and 12 baseline algorithms in four benchmark datasets. The superiority of our BroadCAE can be observed, compared with other baseline comparative algorithms. Especially our BroadCAE gets the highest score of $\bar{F}1$ and $\bar{R}$ in all four datasets. However, the lower $\bar{P}$ is recorded in the MBD (84.44%), SMD (94.90%), and ASD (74.46%). The imbalance between anomaly and normal samples results in this situation. The proportion of correct identification of normal samples is larger than that of abnormal samples. The confusion matrices of our BroadCAE detection results are displayed in Fig. 6, which provides more details. As for EMOS, our BroadCAE achieves the best $\bar{P}$ (91.76%), which ascribes to the less false negative samples.

In the next step, we further discuss comparative anomaly detection algorithms. It is noteworthy that the deep encoder-decoder-based methods always outperform the traditional machine-learning-based anomaly detection algorithms. For example, Bagel attains the best $\bar{P}$ in MBD (93.25%), SMD (96.16%), and ASD (83.62%), although the quantity of predicted anomaly samples is smaller. The CAE-M and Anomaly

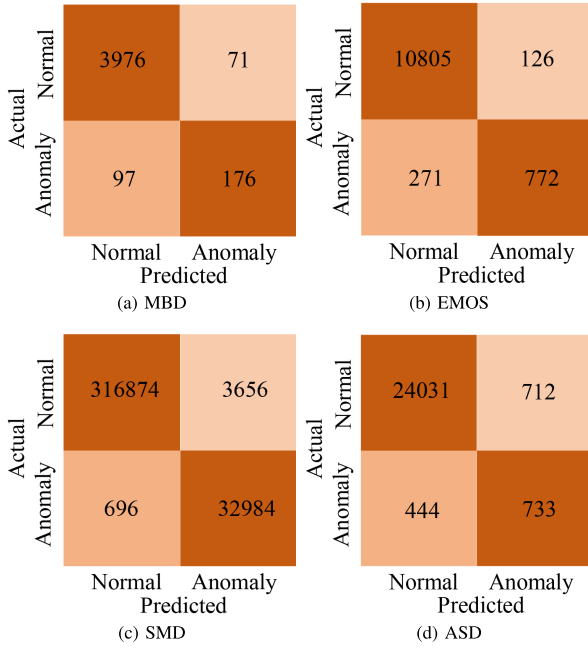


Fig. 6. Confusion matrices on four datasets. The normal and anomaly samples belong to the positive and negative classes, respectively. A majority of normal and anomaly samples are well identified, which results in the higher $F1$.

Transformer acquire the second-best $\bar{F}1$ and $\bar{R}$ in SMD and ASD, respectively. For the EMOS, USAD attains the second-best $\bar{F}1$ (84.06%) and $\bar{P}$ (84.77%), which SaVAE-SR get the second-best $\bar{R}$ (84.94%).

To illustrate the efficiency, we select six competitive algorithms in detection accuracy, including Bagel, USAD, SaVAE-SR, TranAD, Anomaly Transformer, and CAE-M. The averaged training time, averaged testing time, and averaged memory used by these algorithms are regarded as baseline results. As we know from Fig. 7, our BroadCAE does not outperform the baseline about efficiency. Although the training procedure of BroadCAE takes less time, BroadCAE needs more testing time and memory in SMD and ASD. This ascribes to the broad network-based threshold selection.

RQ2: How about the impact of parameters in BroadCAE?

The parameter always has an impact on BroadCAE's performance. The following parameters are discussed for answering this RQ.

- Dimension of latent variable.
- Number of feature mapped nodes.
- Number of enhancement nodes.
- Anomaly bias.
- Normal bias.

We follow the advantage of the encoder-decoder architecture to develop BroadCAE. The dimension of the latent variable is involved with the feature extractor. If the dimension is larger enough, information is abundant, and parameter tuning becomes time-consuming. In contrast, if it is in a lower-dimensional space, limited features will be modeled, and encoder representation may be invalidated. We apply one more

performance metric, area under the ROC curve (AUC), to evaluate the impact of the latent variable dimensions. Fig. 8 presents the experiment results of all the experimental datasets, which are achieved by different dimensions. We start from 32 dimensions, keeping the interval of 32 and up to 320 dimensions. As we increase the dimensions, the performance of BroadCAE does not improve by a larger margin. For example, the detection results about four evaluation metrics become poorer in MBD. As for SMD, the value of $\bar{F}1$, $\bar{R}$, and AUC are relatively steady. It is noteworthy that the AUC of MBD, EMOS, and ASD have dropped. However, the $\bar{F}1$ of EMOS and ASD do not descend, which ascribes to dynamic threshold selection.

The remaining parameters come from the dynamic threshold selection, of which the first two are related to broad network learning, and the others form the supervised pseudo threshold. Because the AUC is always considered for feature representation, we only use $\bar{F}1$, $\bar{R}$, and $\bar{P}$ as performance metrics to evaluate the impact of these parameters.

In Fig. 9, the impact of different numbers of feature mapped nodes is shown. We select up to 50 feature mapped nodes in each experiment, starting from only 5. The $\bar{F}1$ about ASD achieves the maximum values and stops rising. As for MBD, the $\bar{F}1$ also improves with the increase of feature mapped nodes. The $\bar{P}$ of these two datasets keep the same pace as well. In the view of $\bar{R}$, the improvement is weak, although more feature mapped nodes are added. In other words, more network parameters can not bring more powerful detection results. Considering the experiment setting, we recommend that the number of feature mapped nodes is between 25 and 35.

In Fig. 10, we pay attention to the influence of enhancement nodes. The number of enhancement nodes is between 100 and 1000 with an interval of 100. For the $\bar{F}1$, the SMD presents a growing tendency from 100 to 300. The improvement of $\bar{P}$ in the SMD occurs again. For the $\bar{R}$, the MBD also keeps an upward trend. However, the $\bar{P}$ about MMS is in fluctuation. Besides, the EMOS in $\bar{P}$ reaches the peak and then begins to drop in the next stage. We deem the increasing feature mapped node is not good at broad network generalization for learning threshold patterns. The overfitting still exists. Considering the actual setting, we recommend that the number of enhancement nodes should be under 500.

In Fig. 11, we analyze the effect of different anomaly biases. Before the anomaly bias is 0.2, all of the datasets reach their peak of $\bar{F}1$. After that, the ASD decreases. In the meanwhile, MBD, EMOS, and SMD are still at the top. With the decreasing of anomaly bias, the increase of $\bar{R}$ in four benchmark datasets is in the range of 2% to 8%. Inversely, the $\bar{P}$ is down by about averaged 3% with the smaller anomaly bias. The anomaly bias should be between 0.01 and 0.1.

Fig. 12 displays how the normal bias impacts the performance of BroadCAE. The value of normal bias starts from 1.2 and is up to 3 with an interval of 0.2. For the $\bar{F}1$, MBD, SMD, and ASD present a growing tendency. The EMOS achieves a peak and then decreases. The EMOS, MBD, and ASD are at the peak of $\bar{R}$ while the normal bias is 1.4.

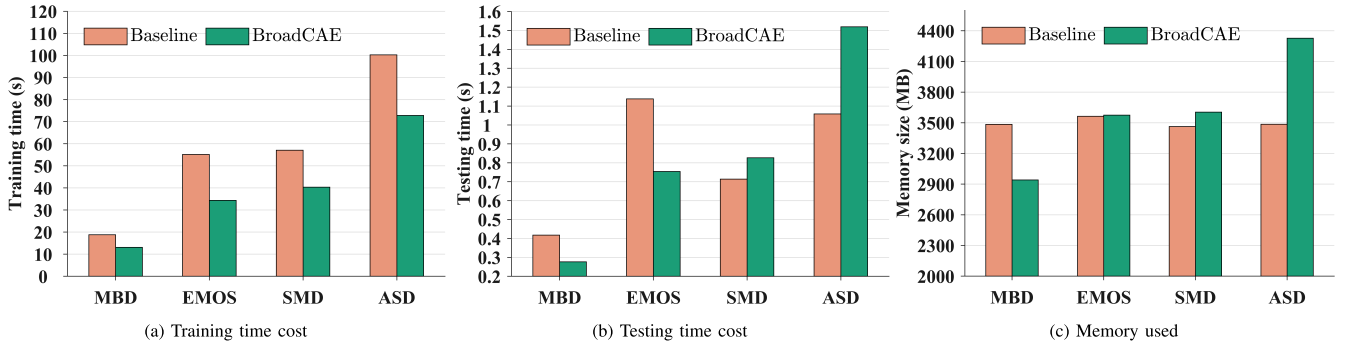


Fig. 7. Training time, testing time, and memory consumption of BroadCAE and baseline algorithm.

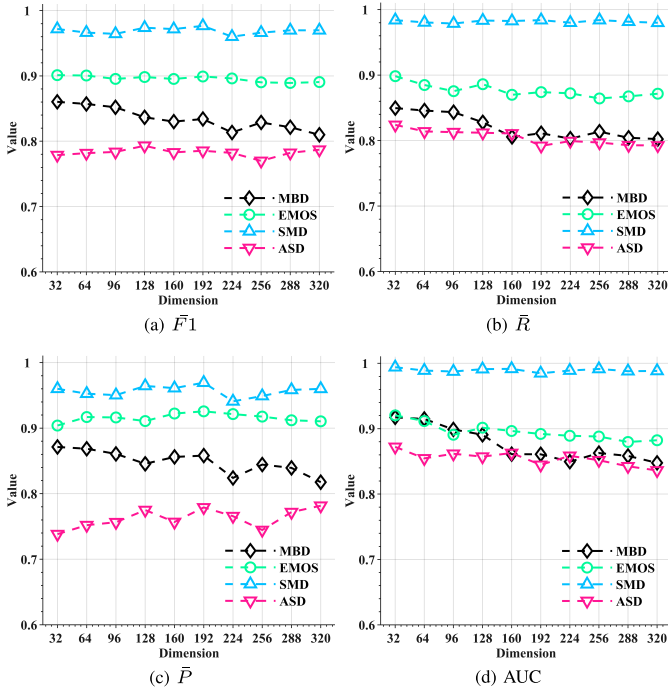


Fig. 8. Impact of latent variable dimension.

After the best normal bias, they fall sharply, which leads to the decrease of EMOS's $\bar{F}1$. Similar to the $\bar{F}1$, $\bar{P}$ keeps an upward trend with the larger normal bias. We suggest that the normal bias is between 1.8 and 2.8.

RQ3: How about the effectiveness of each component in BroadCAE?

Our design of BroadCAE is to optimize the feature representation and add the dynamic threshold selection. The reconstruction loss L_r is replaced by contrastive loss L_c , which extracts both intra-class and inter-class features in representation. The dynamic threshold selection generates the different thresholds for different testing samples, instead of the fixed value. To analyze the effect of the above-mentioned components, we conduct ablation experiments.

1) *Effect of contrastive loss.* The experimental results are displayed in Fig. 13. The $\bar{F}1$, $\bar{R}$, $\bar{P}$, and AUC are referred to as performance metrics. The brown “ L ” denotes the combination of “ L_r ” and “ L_c ”. The green “ L_r ” and

TABLE V
THE EFFECTIVENESS OF DYNAMIC THRESHOLD SELECTION

Dataset	Metric	Fixed A	Fixed B	Dynamic	Improvement
MBD	$\bar{F}1$	75.48	72.86	82.87	10.01%
	$\bar{R}$	63.81	78.58	81.36	2.78%
	$\bar{P}$	92.38	67.92	84.44	/
	Acc	95.23	91.23	96.11	0.88%
EMOS	$\bar{F}1$	75.95	80.83	89.02	8.19%
	$\bar{R}$	65.23	83.55	86.43	2.88%
	$\bar{P}$	90.88	78.28	91.76	0.88%
	Acc	93.61	93.25	96.68	3.07%
SMD	$\bar{F}1$	71.22	92.16	96.62	4.46%
	$\bar{R}$	57.11	95.87	98.40	2.53%
	$\bar{P}$	94.60	88.73	94.90	0.3%
	Acc	91.81	96.95	98.77	1.82%
ASD	$\bar{F}1$	76.61	76.45	76.98	0.37%
	$\bar{R}$	68.37	80.17	79.70	/
	$\bar{P}$	87.12	73.07	74.46	/
	Acc	95.35	95.20	95.54	0.19%

red “ L_c ” indicate the reconstruction loss and contrastive loss, respectively. For $\bar{F}1$, $\bar{P}$, and AUC, the “ L_c ” outperforms the “ L_r ” and “ L ”. However, the $\bar{R}$ of “ L_c ” in MBD does not attain the improvement. The reason for this phenomena is that broad-learning-based threshold selection can cover the shortcoming of feature representation. Besides, in some benchmark datasets, the result of “ L ” is improved by a margin, which illustrates that the inter-class feature is effective.

2) *Effect of dynamic threshold selection.* This part is directly related to the identification of anomalies. The $\bar{F}1$, $\bar{R}$, $\bar{P}$ are used. In addition, we adopt detection accuracy (Acc), a performance metric about the ground truth of normal and anomaly samples in testing data. Table V displays the experiment results. The “Dynamic” is the dynamic threshold selection in our BroadCAE. The “Fixed A” denotes the threshold that comes from the best result in the ROC curve. The “Fixed B” denotes the 3-sigma criterion. Compared to the “Fixed A” and “Fixed B”, the new “Dynamic” sees the maximum 10.01% jump in $\bar{F}1$ and the maximum 3.07% leap in Acc. From this reference’s view [46], broad learning is more

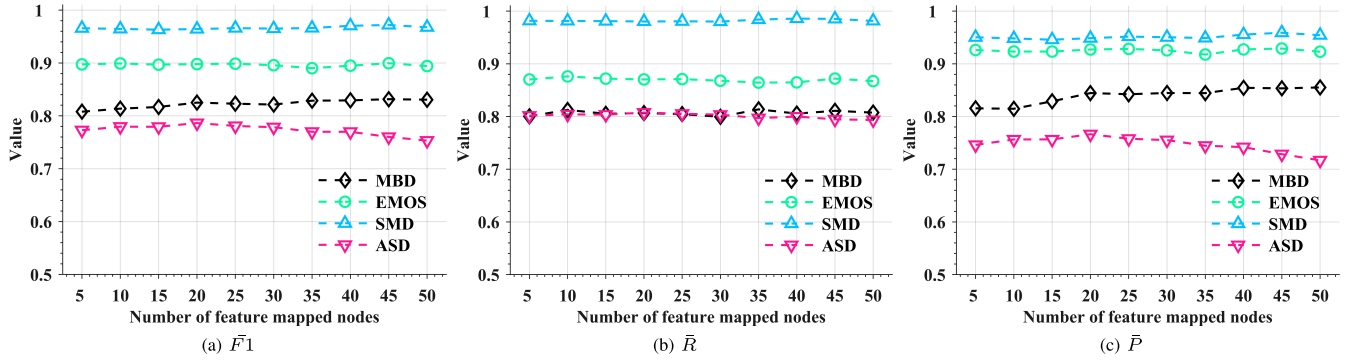


Fig. 9. Impact of the number of feature mapped nodes.

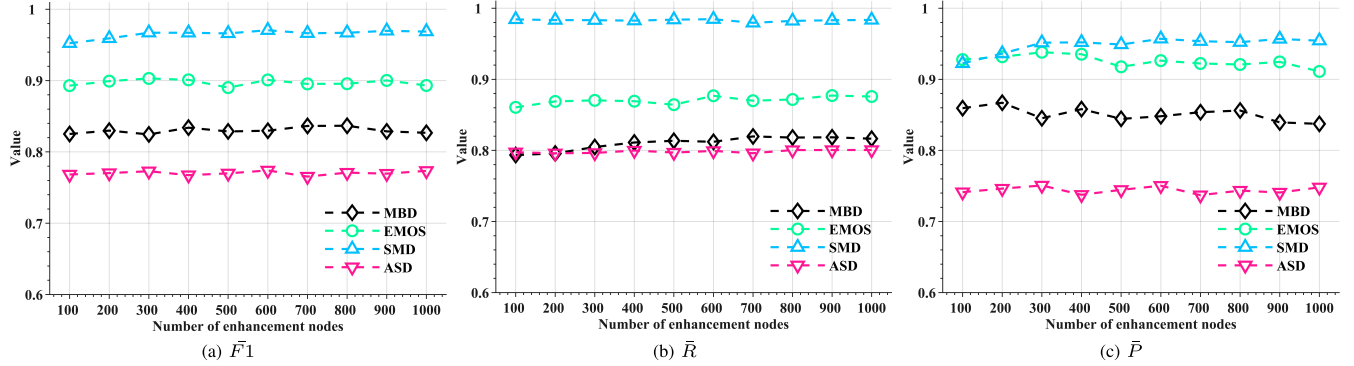


Fig. 10. Impact of the number of enhancement nodes.

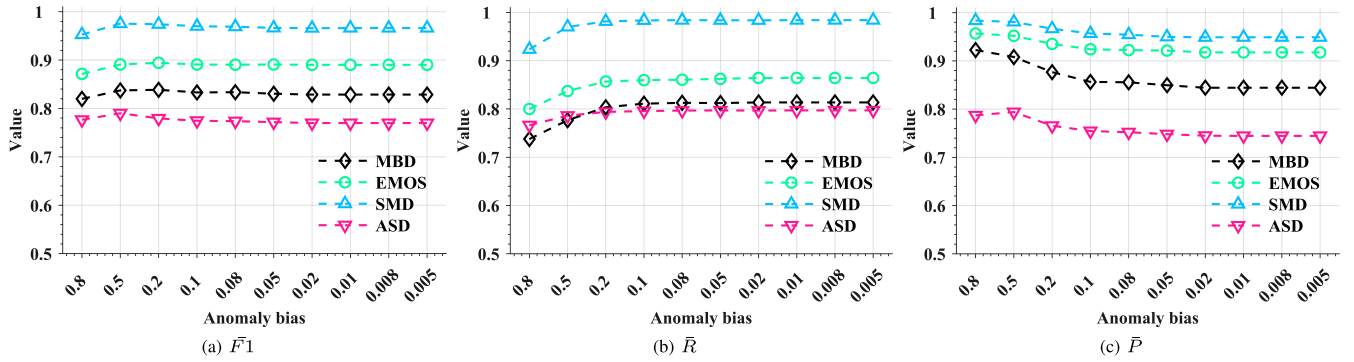


Fig. 11. Impact of anomaly bias.

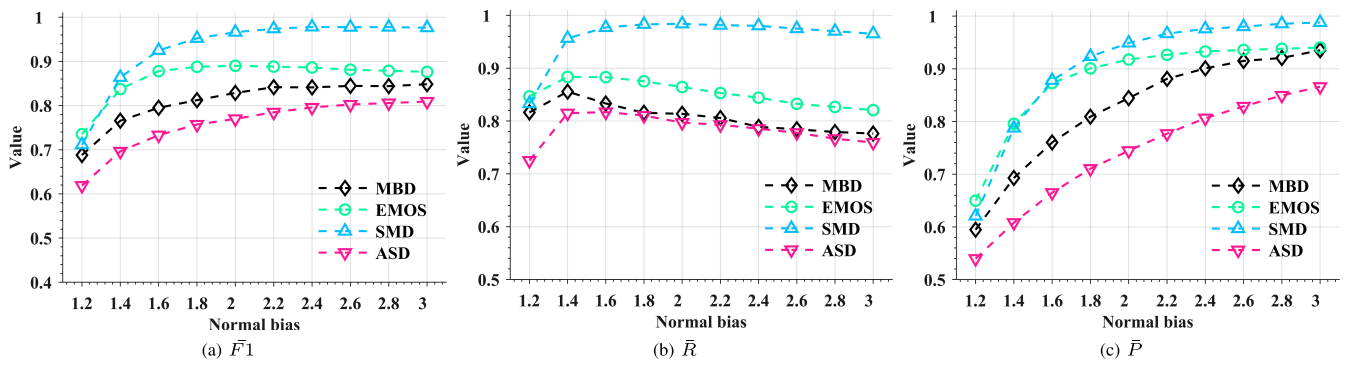


Fig. 12. Impact of normal bias.

suitable to the online setting. In addition, we do not need to consider the selection and combination of deep network blocs while adopting broad learning. This is the reason why we

recommend broad-learning-based dynamic threshold selection. In addition, for ASD, the “Dynamic” only acquires the 0.37% improvement of $\bar{F}1$.

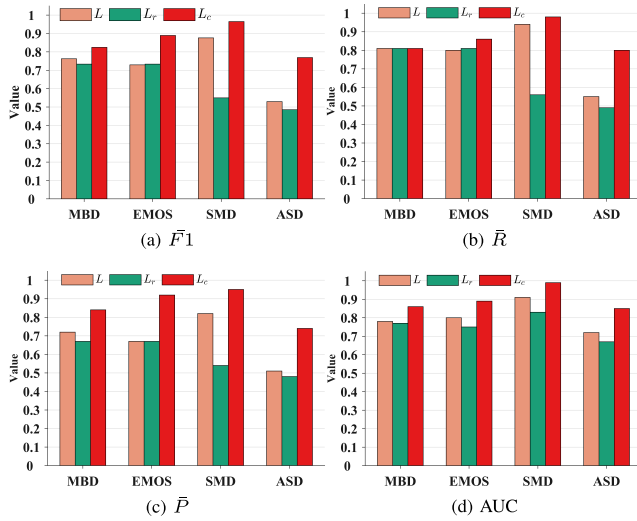


Fig. 13. Effectiveness of contrastive loss.

V. CONCLUSION

In this paper, we propose the cloud anomaly detector, BroadCAE, which incorporates contrastive learning and broad network under the autoencoder framework. The contrastive loss enriches the representation by extracting additional intra-class and inter-class features. The cloud anomaly and normal sample are equally exploited. The broad network-based dynamic threshold selection aims at the changing cloud environment. The learned threshold pattern provides the dynamic threshold, instead of the fixed one. The experimental results in four datasets illustrate that our BroadCAE outperforms the comparative method, and the improved components of BroadCAE are practical.

For cloud operation, the accuracy of the anomaly detector is essential to address the fault. The BroadCAE can be regarded as one of the state-of-the-art methods. In future works, we will explore the following research points for application-driven anomaly detection:

- 1) the classification of anomaly or system fault,
- 2) the location and cause root of anomaly or system fault.

REFERENCES

- [1] H. Jayatilaka, C. Krintz, and R. Wolski, "Detecting performance anomalies in cloud platform applications," *IEEE Trans. Cloud Comput.*, vol. 8, no. 3, pp. 764–777, Jul.–Sep. 2020.
- [2] D. Saxena, I. Gupta, A. K. Singh, and C.-N. Lee, "A fault tolerant elastic resource management framework toward high availability of cloud services," *IEEE Trans. Netw. Service Manag.*, vol. 19, no. 3, pp. 3048–3061, Sep. 2022.
- [3] M. Soualhia, F. Khomh, and S. Tahar, "A dynamic and failure-aware task scheduling framework for Hadoop," *IEEE Trans. Cloud Comput.*, vol. 8, no. 2, pp. 553–569, Apr.–Jun. 2020.
- [4] M. Zoure, T. Ahmed, and L. Réveillère, "Network services anomalies in NFV: Survey, taxonomy, and verification methods," *IEEE Trans. Netw. Service Manag.*, vol. 19, no. 2, pp. 1567–1584, Jun. 2022.
- [5] T. Wang, W. Zhang, J. Xu, and Z. Gu, "Workflow-aware automatic fault diagnosis for microservice-based applications with statistics," *IEEE Trans. Netw. Service Manag.*, vol. 17, no. 4, pp. 2350–2363, Dec. 2020.
- [6] S. Kardani-Moghaddam, R. Buyya, and K. Ramamohanarao, "ADRL: A hybrid anomaly-aware deep reinforcement learning-based resource scaling in clouds," *IEEE Trans. Parallel Distrib. Syst.*, vol. 32, no. 3, pp. 514–526, Mar. 2021.
- [7] L. Yang and A. Shami, "A multi-stage automated online network data stream analytics framework for IIoT systems," *IEEE Trans. Ind. Inform.*, vol. 19, no. 2, pp. 2107–2116, Feb. 2023.
- [8] G. Zhong, F. Liu, J. Jiang, and C. L. P. Chen, "CauseFormer: Interpretable anomaly detection with stepwise attention for cloud service," *IEEE Trans. Netw. Service Manag.*, early access, Jul. 31, 2023, doi: 10.1109/TNSM.2023.3299846.
- [9] Z. Zou, Y. Xie, K. Huang, G. Xu, D. Feng, and D. Long, "A docker container tsm-zhong-3353772 isolation forest," *IEEE Trans. Cloud Comput.*, vol. 10, no. 1, pp. 134–145, Jan.–Mar. 2022.
- [10] Y. Song, R. Xin, P. Chen, R. Zhang, J. Chen, and Z. Zhao, "Identifying performance anomalies in fluctuating cloud environments: A robust correlative-GNN-based explainable approach," *Future Gener. Comput. Syst.*, vol. 145, pp. 77–86, Aug. 2023.
- [11] J. Soldani and A. Brogi, "Anomaly detection and failure root cause analysis in (micro) service-based cloud applications: A survey," *ACM Comput. Surv.*, vol. 55, no. 3, pp. 1–39, 2023.
- [12] Y. Chen et al., "Outage prediction and diagnosis for cloud service systems," in *Proc. World Wide Web Conf.*, 2019, pp. 2659–2665.
- [13] G. Fan, L. Chen, H. Yu, and D. Liu, "Modeling and analyzing dynamic fault-tolerant strategy for deadline constrained task scheduling in cloud computing," *IEEE Trans. Syst., Man, Cybern., Syst.*, vol. 50, no. 4, pp. 1260–1274, Apr. 2020.
- [14] Y. Liu, Y. Lin, Q. Xiao, G. Hu, and J. Wang, "Self-adversarial variational autoencoder with spectral residual for time series anomaly detection," *Neurocomputing*, vol. 458, pp. 349–363, Oct. 2021.
- [15] M. S. Islam, W. Pourmajidi, L. Zhang, J. Steinbacher, T. Erwin, and A. Miranskyy, "Anomaly detection in a large-scale cloud platform," in *Proc. IEEE/ACM 43rd Int. Conf. Softw. Eng. Pract. (ICSE-SEIP)*, 2021, pp. 150–159.
- [16] L. Li, J. Yan, H. Wang, and Y. Jin, "Anomaly detection of time series with smoothness-inducing sequential variational auto-encoder," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 32, no. 3, pp. 1177–1191, Mar. 2021.
- [17] A. Garg, W. Zhang, J. Samaran, R. Savitha, and C.-S. Foo, "An evaluation of anomaly detection and diagnosis in multivariate time series," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 33, no. 6, pp. 2508–2517, Jun. 2022.
- [18] T. Zhao, T. Jiang, N. Shah, and M. Jiang, "A synergistic approach for graph anomaly detection with pattern mining and feature learning," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 33, no. 6, pp. 2393–2405, Jun. 2022.
- [19] G. Zhong, F. Liu, J. Jiang, B. Wang, and C. L. P. Chen, "Refining one-class representation: A unified transformer for unsupervised time-series anomaly detection," *Inf. Sci.*, vol. 656, Jan. 2024, Art. no. 119914.
- [20] Z. Li, W. Chen, and D. Pei, "Robust and unsupervised KPI anomaly detection based on conditional variational autoencoder," in *Proc. IEEE 37th Int. Perform. Comput. Commun. Conf. (IPCCC)*, 2018, pp. 1–9.
- [21] J. Audibert, P. Michiardi, F. Guyard, S. Marti, and M. A. Zuluaga, "USAD: UnSupervised anomaly detection on multivariate time series," in *Proc. 26th ACM SIGKDD Int. Conf. Knowl. Discov. Data Min.*, 2020, pp. 3395–3404.
- [22] J. Xu, H. Wu, J. Wang, and M. Long, "Anomaly transformer: Time series anomaly detection with association discrepancy," in *Proc. Int. Conf. Learn. Represent.*, 2022, pp. 1–20.
- [23] S. Tuli, G. Casale, and N. R. Jennings, "TranAD: Deep transformer networks for anomaly detection in multivariate time series data," *Proc. VLDB Endowment*, vol. 15, no. 6, pp. 1201–1214, 2022.
- [24] Q. Lin et al., "EDITS: An easy-to-difficult training strategy for cloud failure prediction," in *Proc. ACM Web Conf.*, 2023, pp. 371–375.
- [25] B. Li, D. Yu, J. Wu, P. Ju, and Z. Li, "Coordinated cloud-edge anomaly identification for active distribution networks," *IEEE Trans. Cloud Comput.*, vol. 11, no. 2, pp. 1204–1216, Apr.–Jun. 2023.
- [26] F. Schmidt et al., "IFTM-unsupervised anomaly detection for virtualized network function services," in *Proc. IEEE Int. Conf. Web Services (ICWS)*, 2018, pp. 187–194.
- [27] L. Yang et al., "Multi-perspective content delivery networks security framework using optimized unsupervised anomaly detection," *IEEE Trans. Netw. Service Manag.*, vol. 19, no. 1, pp. 686–705, Mar. 2022.
- [28] M. A. Mukwevho and T. Celik, "Toward a smart cloud: A review of fault-tolerance methods in cloud systems," *IEEE Trans. Services Comput.*, vol. 14, no. 2, pp. 589–605, Mar./Apr. 2021.
- [29] S. He, J. Zhu, P. He, and M. R. Lyu, "Experience report: System log analysis for anomaly detection," in *Proc. 27th IEEE Int. Symp. Softw. Rel. Eng.*, 2016, pp. 207–218.

- [30] M. Smara, M. Aliouat, A.-S. K. Pathan, and Z. Aliouat, "Acceptance test for fault detection in component-based cloud computing and systems," *Future Gener. Comput. Syst.*, vol. 70, pp. 74–93, May 2017.
- [31] Q. Lin, H. Zhang, J.-G. Lou, Y. Zhang, and X. Chen, "Log clustering based problem identification for online service systems," in *Proc. IEEE/ACM 38th Int. Conf. Softw. Eng., Companion (ICSE-Companion)*, 2016, pp. 102–111.
- [32] N. Zhao, J. Zhu, R. Liu, D. Liu, M. Zhang, and D. Pei, "Label-less: A semi-automatic labelling tool for KPI anomalies," in *Proc. IEEE INFOCOM Conf. Comput. Commun.*, 2019, pp. 1882–1890.
- [33] T. Huang et al., "An LOF-based adaptive anomaly detection scheme for cloud computing," in *Proc. IEEE 37th Annu. Comput. Softw. Appl. Conf. Workshops (COMPSAC)*, 2013, pp. 206–211.
- [34] J. Álvarez Cid-Fuentes, C. Szabo, and K. Falkner, "Adaptive performance anomaly detection in distributed systems using online SVMs," *IEEE Trans. Dependable Secure Comput.*, vol. 17, no. 5, pp. 928–941, Sep./Oct. 2020.
- [35] N. Zhao et al., "Understanding and handling alert storm for online service systems," in *Proc. IEEE/ACM 42nd Int. Conf. Softw. Eng., Softw. Eng. Pract. (ICSE-SEIP)*, 2020, pp. 162–171.
- [36] M. M. Breunig, H.-P. Kriegel, R. T. Ng, and J. Sander, "LOF: Identifying density-based local outliers," in *Proc. Int. Conf. Manag. Data (SIGMOD)*, 2000, pp. 93–104.
- [37] D. M. J. Tax and R. P. W. Duin, "Support vector data description," *Mach. Learn.*, vol. 54, no. 1, pp. 45–66, 2004.
- [38] S. Hawkins, H. He, G. Williams, and R. Baxter, "Outlier detection using replicator neural networks," in *Proc. 4th Int. Conf. Data Ware. Knowl. Discov.*, 2002, pp. 170–180.
- [39] H. Xu et al., "Unsupervised anomaly detection via variational auto-encoder for seasonal KPIs in Web applications," in *Proc. World Wide Web Conf.*, 2018, pp. 187–196.
- [40] D. P. Kingma and M. Welling, "Auto-encoding variational Bayes," in *Proc. Int. Conf. Learn. Represent.*, 2014, pp. 1–14.
- [41] D. Liu et al., "Opprentice: Towards practical and automatic anomaly detection through machine learning," in *Proc. ACM Int. Meas. Conf.*, 2015, pp. 211–224.
- [42] K. Sohn, H. Lee, and X. Yan, "Learning structured output representation using deep conditional generative models," in *Proc. Adv. Neural Inf. Process. Syst.*, 2015, pp. 3483–3491.
- [43] S. Lin, R. Clark, R. Birke, S. Schönborn, N. Trigoni, and S. Roberts, "Anomaly detection for time series using VAE-LSTM hybrid model," in *Proc. IEEE Int. Conf. Acoust., Speech Signal*, 2020, pp. 4322–4326.
- [44] A. Vaswani et al., "Attention is all you need," in *Proc. 31st Adv. Neural Inf. Process. Syst.*, 2017, pp. 5998–6008.
- [45] C. L. P. Chen and Z. Liu, "Broad learning system: An effective and efficient incremental learning system without the need for deep architecture," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 29, no. 1, pp. 10–24, Jan. 2018.
- [46] C. L. P. Chen, Z. Liu, and F. Shuang, "Universal approximation capability of broad learning system and its structural variations," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 30, no. 4, pp. 1191–1204, Apr. 2019.
- [47] S. Feng and C. L. P. Chen, "Fuzzy broad learning system: A novel neuro-fuzzy model for regression and classification," *IEEE Trans. Cybern.*, vol. 50, no. 2, pp. 414–424, Feb. 2020.
- [48] Z. Liu, C. L. P. Chen, S. Feng, Q. Feng, and T. Zhang, "Stacked broad learning system: From incremental flatted structure to deep model," *IEEE Trans. Syst., Man, Cybern., Syst.*, vol. 51, no. 1, pp. 209–222, Jan. 2021.
- [49] T. Song, W. Zheng, P. Song, and Z. Cui, "EEG emotion recognition using dynamical graph convolutional neural networks," *IEEE Trans. Affect. Comput.*, vol. 11, no. 3, pp. 532–541, Jul.–Sep. 2020.
- [50] T. Qiu, X. Liu, X. Zhou, W. Qu, Z. Ning, and C. L. P. Chen, "An adaptive social spammer detection model with semi-supervised broad learning," *IEEE Trans. Knowl. Data Eng.*, vol. 34, no. 10, pp. 4622–4635, Oct. 2022.
- [51] F. Zhou and C. Cao, "Overcoming catastrophic forgetting in graph neural networks with experience replay," in *Proc. 35th AAAI Conf. Artif. Intell.*, 2021, pp. 4714–4722.
- [52] Z. He et al., "A spatiotemporal deep learning approach for unsupervised anomaly detection in cloud systems," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 34, no. 4, pp. 1705–1719, Apr. 2023.
- [53] Y. Su, Y. Zhao, C. Niu, R. Liu, W. Sun, and D. Pei, "Robust anomaly detection for multivariate time series through stochastic recurrent neural network," in *Proc. 25th ACM SIGKDD Int. Conf. Knowl. Discov. Data Min.*, 2019, pp. 2828–2837.
- [54] Z. Li et al., "Multivariate time series anomaly detection and interpretation using hierarchical inter-metric and temporal embedding," in *Proc. 27th ACM SIGKDD Int. Conf. Knowl. Discov. Data Min.*, 2021, pp. 3220–3230.
- [55] V. Nair and G. E. Hinton, "Rectified linear units improve restricted Boltzmann machines," in *Proc. 27rd Int. Conf. Int. Conf. Mach. Learn.*, 2010, pp. 807–814.
- [56] S. Ioffe and C. Szegedy, "Batch normalization: Accelerating deep network training by reducing internal covariate shift," in *Proc. 32rd Int. Conf. Mach. Learn.*, 2015, pp. 448–456.
- [57] D. P. Kingma and J. Ba, "ADAM: A method for stochastic optimization," in *Proc. Int. Conf. Learn. Represent.*, 2015, pp. 1–15.
- [58] J. Davis and M. Goadrich, "The relationship between precision-recall and ROC curves," in *Proc. 23rd Int. Conf. Mach. Learn.*, 2006, pp. 233–240.
- [59] M. Amer, M. Goldstein, and S. Abdennadher, "Enhancing one-class support vector machines for unsupervised anomaly detection," in *Proc. ACM SIGKDD Workshop Outl. Detec. Descr.*, 2013, pp. 8–15.
- [60] F. T. Liu, K. M. Ting, and Z.-H. Zhou, "Isolation-based anomaly detection," *ACM Trans. Knowl. Discov. Data*, vol. 6, no. 1, pp. 1–39, 2012.
- [61] Y. Zhang, Y. Chen, J. Wang, and Z. Pan, "Unsupervised deep anomaly detection for multi-sensor time-series signals," *IEEE Trans. Knowl. Data Eng.*, vol. 35, no. 2, pp. 2118–2132, Feb. 2023.
- [62] I. J. Goodfellow et al., "Generative adversarial nets," in *Proc. Adv. Neural Inf. Process. Syst.*, 2014, pp. 2672–2680.



Guoxiang Zhong received the M.S. degree from the Guangdong University of Technology, Guangzhou, China, in 2020. He is currently pursuing the Ph.D. degree with the School of Computer Science and Engineering, South China University of Technology, Guangzhou. His research interests include cloud computing and data mining.



Fagui Liu (Member, IEEE) received the M.S. degree from Beihang University in 1991, and the Ph.D. degree from the South China University of Technology, China, in 2006, where she is currently a Professor with the School of Computer Science and Engineering, South China University of Technology. She is also with the Department of New Networks, Peng Cheng Laboratory, Shenzhen. Her current research interests include service computing, Internet of Things, cloud computing, and big data.



Jun Jiang (Member, IEEE) received the Ph.D. degree in computer science and technology from the South China University of Technology, Guangzhou, China, in 2022. He was a visiting student with Peng Cheng Laboratory, Shenzhen, China, in 2023. He is currently a Lecturer with the College of Information Science and Technology and the College of Artificial Intelligence, Nanjing Forestry University, Nanjing, China. His research interests include data stream classification, anomaly detection, cloud computing, and Internet of Things.



Bin Wang received the B.S. and Ph.D. degrees from the South China University of Technology in 2014 and 2021, respectively. He is currently a Research Scientist with the Department of New Networks, Peng Cheng Laboratory, Shenzhen. His research interests include cloud computing, edge computing, and energy efficiency.



C. L. Philip Chen (Fellow, IEEE) is the Chair Professor and the Dean of the College of Computer Science and Engineering, South China University of Technology, Guangzhou, China. Been a Program Evaluator of the Accreditation Board of Engineering and Technology Education, USA, for computer engineering, electrical engineering, and software engineering programs, he successfully architects the University of Macau's Engineering and Computer Science programs receiving accreditations from Washington/Seoul Accord through the Hong Kong Institute of Engineers, of which is considered as his utmost contribution in engineering/computer science education for Macau as the Former Dean of the Faculty of Science and Technology. His current research interests include cybernetics, computational intelligence, and systems.

Dr. Chen received the IEEE Norbert Wiener Award in 2018 for his contribution in systems and cybernetics, and machine learnings and the IEEE Joseph Wohl Award in 2021. He received two times Best Transactions Paper Award from IEEE TRANSACTIONS ON NEURAL NETWORKS AND LEARNING SYSTEMS for his papers in 2014 and 2018. He was a recipient of the 2016 Outstanding Electrical and Computer Engineers Award from his alma mater, Purdue University, West Lafayette, IN, USA, in 1988, after he graduated from the University of Michigan at Ann Arbor, Ann Arbor, MI, USA, in 1985. He is a Highly Cited Researcher by Clarivate Analytics from 2018 to 2021. He is a Fellow of the American Association for the Advancement of Science, the International Association for Pattern Recognition, the Chinese Association of Automation, and HKIE, and a member of the Academia Europaea and the European Academy of Sciences and Arts. He was the Editor-in-Chief of the IEEE TRANSACTIONS ON CYBERNETICS from 2020 to 2021 after he completed his term as the Editor-in-Chief of the IEEE TRANSACTIONS ON SYSTEMS, MAN, AND CYBERNETICS: SYSTEMS from 2014 to 2019, followed by serving as the IEEE Systems, Man, and Cybernetics Society President from 2012 to 2013. He is currently serving as the Deputy Director for *CAAI Transactions on Artificial Intelligence*, an Associate Editor for the IEEE TRANSACTIONS ON ARTIFICIAL INTELLIGENCE, IEEE TRANSACTIONS ON SYSTEMS, MAN, AND CYBERNETICS: SYSTEMS, IEEE TRANSACTIONS ON FUZZY SYSTEMS, and *China Sciences: Information Sciences*.



Xi Yao received the B.S. degree from the South China University of Technology, Guangzhou, China, in 2023. She will pursue the Ph.D. degree with the School of Computer Science and Engineering, South China University of Technology. Her research interests include anomaly detection and cloud computing.